

Gate Matrix Layout

OMAR WING, FELLOW, IEEE, SHUO HUANG, AND RUI WANG

Abstract—A graph-theoretic description of the problem of layout of CMOS circuits in the style of gate matrix in minimum area is presented. The problem is formulated as one of finding two assignment functions f and h such that the layout $L(f, h)$ requires the minimum number of rows of the gate matrix. The function f maps the distinct gates of the transistors to the columns of the gate matrix and the function h maps the nets of the circuit to the rows such that all of the vertical diffusion runs which connect nets on different rows are realizable. A two-stage approach to the problem is described which first obtains a layout without regard to the vertical constraints and then rows are permuted to satisfy the constraints. Detailed algorithms and examples are given. The gate matrix layout of a 118-transistor circuit was obtained in 9 s on a mainframe computer.

I. INTRODUCTION

WE CONSIDER the problem of layout of static CMOS circuits in the style of gate matrix [1] in minimum area. In an earlier paper [2], we described an interval-graph-based algorithm to obtain a symbolic layout. In this paper we give a fuller description of the layout problem taking into account certain practical constraints which the earlier paper did not consider. A layout program has been implemented and typical results are that the symbolic layout of the n-channel part of a 118-, 232-, and 306-transistor circuit was obtained in 6, 25, and 15 s, respectively, on a DEC 20 system.

In the gate matrix style, all the p-transistors are placed in the top half of the matrix and all the n-transistors in the lower half. The columns of the matrix are equally spaced polysilicon stripes, each serving two functions: as a transistor gate when diffusion areas are placed on each side of the stripe, and as a conductor which connects the two gates of each complementary pair of transistors. Transistors are connected to one another by segments of metal lines which are placed in the rows of the matrix. In addition, short vertical diffusion stripes are sometimes necessary to connect the drains and sources of transistors to metal lines on different rows. An example is shown in Fig. 1.

In formulating the problem of optimum layout, we shall consider the n-transistors only since the p-part of the circuit is determined once the topology of the n-part is known. Secondly, we shall ignore the connections to V_{SS} (ground). We assume they will be placed on a second layer of metal and their routing will not be discussed in this paper.

Manuscript received February 2, 1985; revised April 9, 1985. This work was supported by the Semiconductor Research Corporation under Contract 83-01-002.

The authors are with the Department of Electrical Engineering, Columbia University, New York, NY 10027.

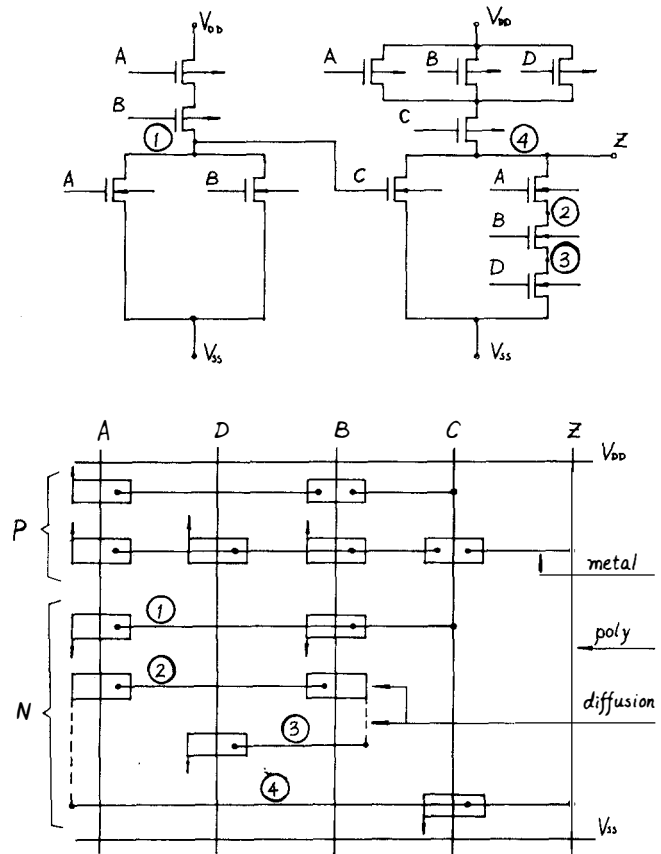


Fig. 1. Gate matrix layout of a CMOS circuit.

II. FORMULATION OF PROBLEM

Let $T = \{t_i | t_i \text{ is a transistor}\}$ be the set of transistors. Label the transistor gates with distinct names except that two or more gates connected to the same node will have the same name. We assign a poly stripe to each distinct transistor gate and to each output terminal of the circuit and define the set $G = \{g_i | g_i \text{ is a distinct transistor gate or an output terminal}\}$. The poly stripes will be referred to as *gate lines*. Let $C = \{c_i\}$ be the set of columns of the gate matrix and $R = \{r_i\}$ be the set of rows. In a layout, let $f: G \rightarrow C$ be the function that assigns the gates to the columns. We call the function f the *gate assignment function*.

In a circuit, the transistors are connected to one another at their source, drain and gate. We assign a net to each node of the circuit and denote the set of nets (nodes) by $N = \{n_i\}$. Every transistor (channel) is connected between two nodes (nets) or between a node and ground. Each net

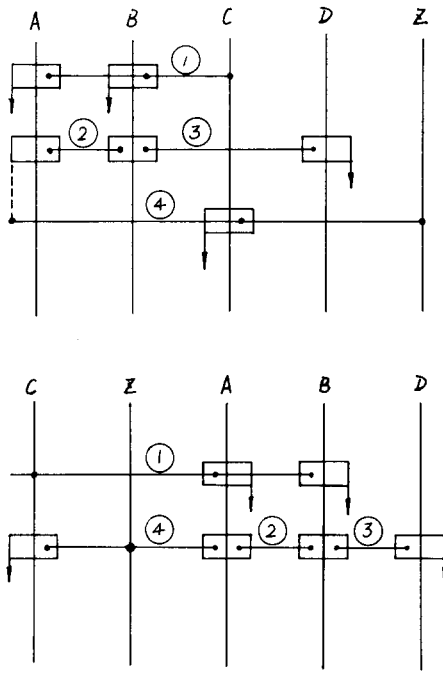


Fig. 2. Two different layouts of the same circuit as Fig. 1 that require fewer rows.

will be realized by a segment of horizontal metal line which is connected to a drain, source, or gate of the transistors of the node. In a layout, we define a *net assignment function* $h: N \rightarrow R$ as the function that assigns the nets to the rows. For each pair (f, h) of functions, there is a layout $L(f, h)$.

Each layout $L(f, h)$ will result in a gate matrix with a certain number of rows. For example, the layout of Fig. 1 requires 4 rows. Fig. 2 shows two layouts which require fewer rows.

Consider a transistor whose drain and source are connected to nets n and m , respectively. In a layout, it is not always possible to place the two nets on the same row. Suppose net n is on the same row as the transistor. Then if net m is on a different row, a short vertical diffusion run will be required to connect m to the source, as exemplified by nets 2 and 3 along gate line B , and by nets 2 and 4 along gate line A in Fig. 1(b). For a given layout $L(f, h)$, for each pair of nets (n, m) such that $h(n) \neq h(m)$, we define the set of diffusion runs $D(n, m) = \{g_k | h(n) \neq h(m) \text{ and there exists a vertical diffusion run between net } n \text{ and net } m \text{ near gate line } k\}$. $D(n, m)$ is said to be *realizable* if each of its vertical diffusion runs can be placed such that it does not conflict with (overlap) any transistor in the same column on a row between row $h(n)$ and row $h(m)$. In Fig. 3(a), $D(1, 2)$ is realizable but $D(1, 3)$ is not. Thus, each function $h: N \rightarrow R$ induces a set of sets of diffusion runs. We say function h is *realizable* if every $D(n, m)$ that it induces is realizable. A layout $L(f, h)$ is *realizable* if h is realizable.

We are now ready to state the optimum gate matrix layout problem, as follows.

Given a set of transistors T together with the set of dis-

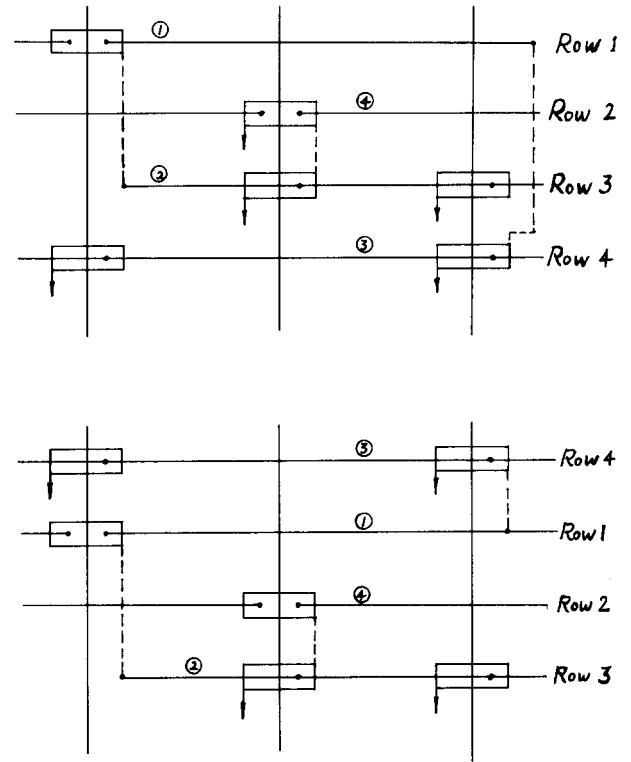


Fig. 3. Permutation of the rows to realize all of the vertical diffusion runs.

tinct gates G and the set of nets N , find a pair of functions $f: G \rightarrow C$ and $h: N \rightarrow R$ such that in the layout $L(f, h)$:

- 1) the number of rows card $[R]$ is minimum; and
- 2) $L(f, h)$ is realizable.

Since the number of columns is fixed by the number of distinct gates, minimization of the number of rows is the same as minimization of the area of the gate matrix.

We propose a two-stage approach to this problem. We first ignore the realizability of the vertical diffusion runs and attempt to find a layout $L(f^*, h^*)$ which results in a minimum number of rows. If h^* is realizable, we are done. If not, starting with h^* , we find a permutation of the rows $P(h^*)$ to obtain a realizable function h (see Fig. 3(b)). If none exists, we propose to enlarge the spacing of some pair of poly stripes to allow the vertical diffusion runs to be placed between them so as not to overlap any transistors. In this way, we keep the number of rows unchanged. Since in practice, the number of columns is much larger than the number of rows, the increase in area by enlarging the space between polys is less than would be the case if the number of rows is increased (assuming one enlarges an equal amount in both direction).

III. ABSTRACTION

Since each transistor is assigned a gate line, it is convenient to regard each net as being "connected" to the gate lines of the transistors of the net. We propose an abstract representation of this relation between gates and nets as follows. Each gate g_j will be represented by a vertical line and each net n_i by a horizontal line segment. Asso-

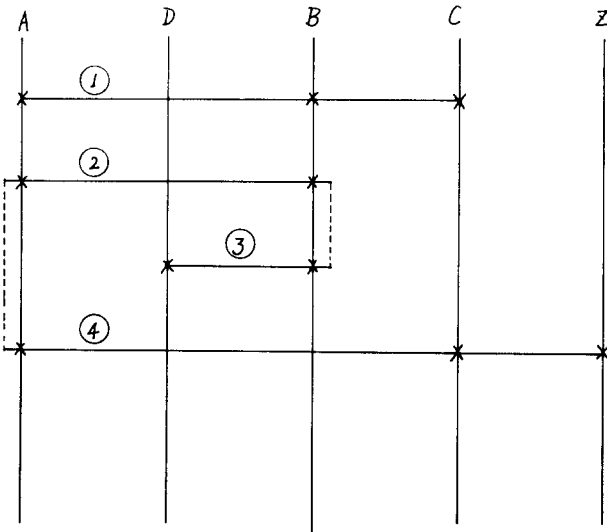


Fig. 4. Abstract representation of a gate matrix layout.

ciated with each net n_i is a *net-gates set* $X(n_i) = \{g_j | n_i \text{ is connected to the drain, source, or gate of a transistor assigned to } g_j\}$. For example, in Fig. 1, $X(1) = \{A, B, C\}$, $X(4) = \{A, C, Z\}$, etc. From a given circuit, the net-gates sets can be found trivially. In the abstract representation, if $g_j \in X(n_i)$, a cross is placed at the intersection of net i and gate j . (A cross indicates a net connection to a transistor, a gate line, or a diffusion run.) Fig. 4 shows a representation of the layout of the n-part of the circuit. The vertical diffusion runs which connect nets on different rows are shown in dotted lines.

Associated with each gate g_j is a *gate-nets set* $Y(g_j) = \{n_i | g_j \in X(n_i)\}$, which lists all the nets connected to gate j . For example, in Fig. 1, $Y(A) = \{1, 2, 4\}$, etc. The gate-nets sets of a circuit can be derived from the net-gates sets trivially.

Thus insofar as gate matrix layout is concerned, a circuit is represented by either the sets $X(n_i)$, $n_i \in N$, or the sets $Y(g_j)$, $g_j \in G$.

Stage one of the layout problem can now be stated as:

Problem 1: Given the net-gates sets $X(n_i)$, $n_i \in N$, of a circuit, find a pair of gate and net assignment functions $f: G \rightarrow C$ and $h: N \rightarrow R$, respectively, such that $\text{card}[R]$ is minimum.

This problem has been considered by Ohtsuki *et al.* [3], [4] and others [9], [10], and formulated as an interval-graph problem, it has been shown to be NP-complete [5].

IV. SERIES-CONNECTED TRANSISTORS

Defining a net as a node of a circuit is convenient, but cumbersome for series-connected transistors. Consider the series-connected transistors A, B, and D of Fig. 1. In the layout three rows are required to place and route the connections of the transistors and the connection to net (node) 4, as shown in Fig. 4. In a static circuit the order in which the three transistors are connected is logically immaterial. This suggests that we treat series-connected transistors as a group and assign one net to represent the group. In ef-

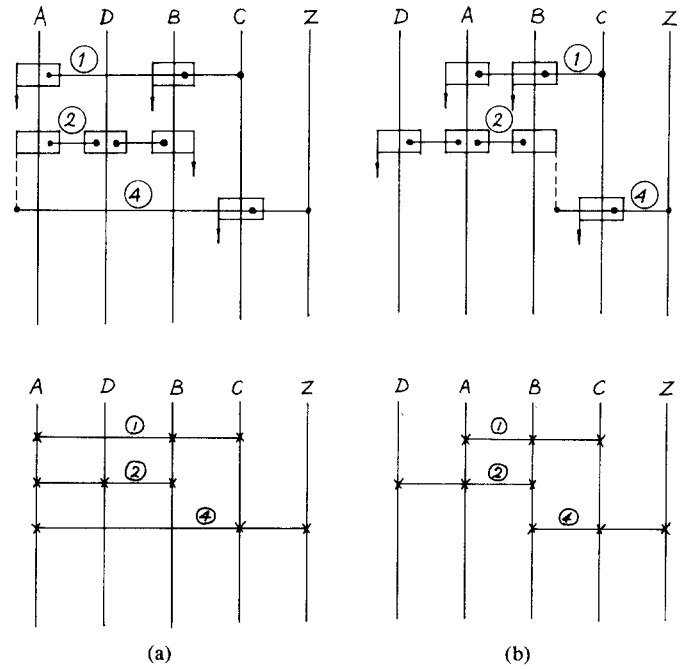


Fig. 5. (a) Series-connected transistors ABD are considered as one net 2. (b) The end transistor of a series group of transistors is redefined.

fect, the internal nodes of the group are merged into one node, which is the net of the group. Thus in Fig. 1, if we represent the group ABD by net 2 and knowing that the order in which the transistors in the group are connected does not matter, we get a layout which requires only two rows to place all the connections of the group, as shown in Fig. 5(a). Moreover, the transistor at either end of the group may be designated as the one which is connected to a node external to the group. Thus in the layout of Fig. 5(b), transistor B of the group is chosen to be connected to the external node 4. This flexibility will be utilized after a gate assignment function is determined.

V. INTERVAL-GRAPH REPRESENTATION

As in a paper by Ohtsuki *et al.* [3], we represent the gate-nets sets by a *connection graph* $H = (V, E)$ where the vertex set $V = \{v_i | n_i \in N\}$ and the edge set $E = \{\langle v_i, v_j \rangle | g_k \text{ such that } n_i \text{ and } n_j \in Y(g_k)\}$. The connection graph can be derived from the sets $Y(g_k)$, $g_k \in G$. This is to say that the connection graph completely describes how the nets are connected to the gates.

In an abstract representation of a layout, the nets can be regarded as intervals which overlap one another. As in [3], we define an *interval graph* associated with a layout $L(f, h)$ as $I(L) = (V, B)$ where V is the same vertex set as in H and the edge set $B = \{\langle v_i, v_j \rangle | n_i \text{ and } n_j \text{ overlap in } L\}$. Fig. 6(a) shows a layout L_a and its interval graph $I(L_a)$ and Fig. 6(b) shows a different layout L_b of the same circuit and its interval graph $I(L_b)$. The connection graph of the circuit is shown in Fig. 6(c). The following are self-evident:

1) Each layout L realizes a connection graph H and the interval graph $I(L)$ contains H as a subgraph.

2) The number of rows in a layout L equals the size of

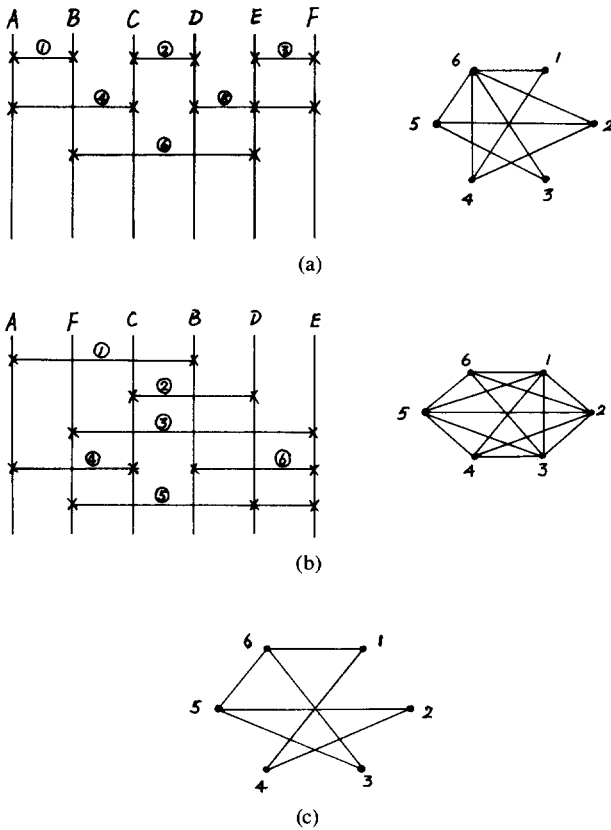


Fig. 6. (a) and (b) Two interval graph representations of two gate matrix layouts. (c) The connection graph of the circuit.

the largest dominant clique of $I(L)$. (A dominant clique is a clique [11] which is not a proper subset of another clique. The size of a clique is the number of vertices it contains.)

3) A lower bound of the minimum number of rows required in any layout is equal to the maximum number of nets connected to any gate, i.e.,

$$\min_L \{\text{card}[R]\} \geq \max_{g_j \in G} \{\text{card}[Y(g_j)]\}.$$

The lower bound can be improved by special treatment of the series-connected transistors as explained in the next section. Problem 1 can now be restated as: Given a connection graph H , find an interval graph I which contain H as a subgraph such that the size of the largest dominant clique of I is minimized.

In [6], we gave an heuristic algorithm to find an interval graph $I \supseteq H$ such that the size of the dominant clique is minimally increased as I is constructed. The algorithm is based on a representation of an interval graph by its dominant clique versus vertex matrix [7] and is detailed in [6] and in Appendix A.

It is shown in [3] and in Appendix B that once an interval graph is found, a gate assignment function f , among many, is determined. In fact, a net assignment function h is also determined. However, because of the flexibility that either end transistor of a series-connected group can be connected to a node external to the group, and for other

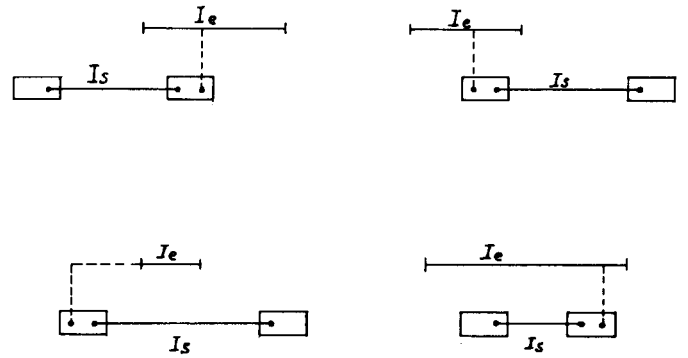


Fig. 7. Identification of the end transistors.

reasons which will be evident shortly, we postpone the determination of the net assignment function until certain net processing has been done, as discussed next.

VI. NET PROCESSING

Suppose a gate assignment function f has been determined as shown in Appendix B so that the column number $f(g_i)$ of each gate line g_i is known. Then the following two net processing steps may be applied.

1) *Identify the End Transistor of a Series Group:* We define an end transistor of a series-connected group of transistors as one which is connected to a node external to the group. In the circuit of Fig. 1, A is an end transistor of the group ABD , which is connected to net 4. We recall that each series-connected group of transistors is represented by one net. Let it be n_s . Then $X(n_s)$ is the set of gates associated with the transistors of the group. Let the external node (net) be n_e . Then $X(n_e)$ is the set of gate lines associated with the transistors connected by n_e . Given a gate sequence specified by the function f , the order in which the gates are arranged is determined.

Let g_{sl} and g_{sr} be the leftmost and rightmost gate of $X(n_s)$, respectively, and similarly for g_{el} and g_{er} . Let $I_s = [g_{sl}, g_{sr}]$ and $I_e = [g_{el}, g_{er}]$ be the intervals of n_s and n_e , respectively. There are four situations.

(a) and (b) If I_e contains g_{sr} (g_{sl}) but not g_{sl} (g_{sr}), the transistor associated with g_{sr} (g_{sl}) is the end transistor. See Fig. 7(a) and (b).

(c) If $I_e \subseteq I_s$, n_e has to be extended to connect to either g_{sl} or g_{sr} . Choose whichever that results in the shortest extension. See Fig. 7(c).

(d) If $I_e \supseteq I_s$, a portion of n_e can be deleted. Choose whichever that results in the longest deletion. See Fig. 7(d).

After the end transistor of a series-connected group is identified, we must update $X(n_e)$, $Y(g_{sr})$ or $Y(g_{sl})$ and $D(n_s, n_e)$.

2) *Merge the Nets:* Consider the situation depicted in Fig. 8. If the intervals representing two nets which have a gate in common overlap at a single gate line which is the common gate, then the two nets can be merged into one, and the connections can be placed in one instead of two rows.

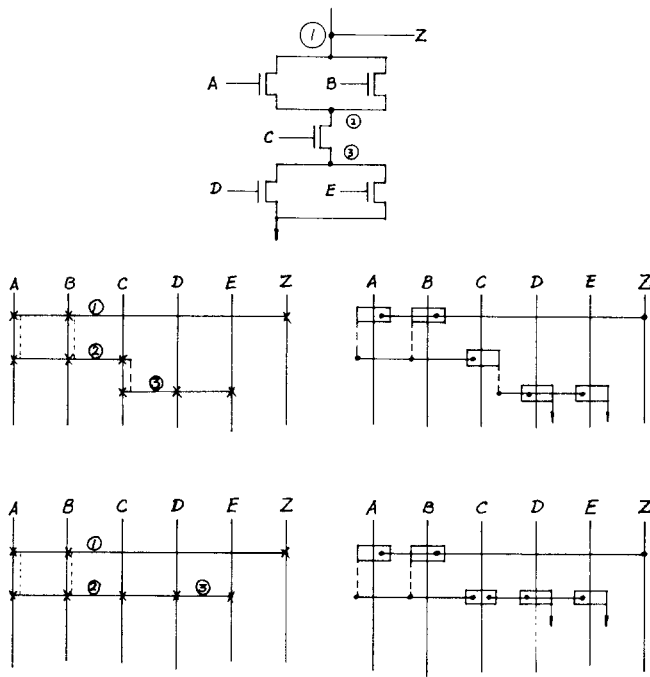


Fig. 8. Merging of nets 2 and 3 into one net.

VII. NET ASSIGNMENT

After changing the end transistors of the series-connected groups of transistors and merging the nets, we apply the left-edge-first algorithm [8] to the vertex versus dominant matrix to find a net assignment function h^* as in [3]. If h^* is realizable, $L(f, h^*)$ is the layout. If not, we proceed to find a realizable net assignment function as discussed next.

VIII. REALIZABLE NET ASSIGNMENT

Consider two nets n_i and n_j for which the set of vertical diffusion runs $D(i, j)$ is not empty. Since diffusion runs should be as short as possible, the two rows to which the nets are assigned, $h(n_i)$ and $h(n_j)$, should be next to each other. However, it is immediately apparent that if there exists a third net n_k for which $D(i, k)$ and $D(j, k)$ are both not empty, the neighboring requirement is cyclic. But if the diffusion runs induced by h are all realizable, then h is acceptable as a solution.

Suppose h^* is not realizable. Let $J(V, E)$ be defined as a row adjacency graph in which $V = \{v_i | v_i \text{ is row } i \text{ in the layout } L(f, h^*)\}$ and $E = \{\langle v_m, v_n \rangle | \text{there exist two nets } m \text{ and } n \text{ such that } D(m, n) \text{ is not empty and that } v_m = h^*(m) \text{ and } v_n = h^*(n)\}$. The graph J , which is derived from h^* , expresses the neighboring requirements among the rows induced by h^* and it may contain cycles.

Our goal is to preserve the number of rows obtained from stage one and to attempt to permute the rows together with the nets assigned to each such that the set of all diffusion runs are realizable, rather than to find another net assignment function which may realize all the diffusion runs but may increase the total number of rows. In terms of the row adjacency graph J , this amounts to find-

ing a permutation of the nodes such that all of the diffusion runs are realizable. This then is the problem of stage two.

The problem of finding a realizable sequence seems very difficult. We propose a procedure to construct a sequence node by node in such a way that as each node is added to the partial sequence, all of the diffusion runs induced by the partial sequence are realized. When no such node can be found, we enlarge the spacing between two poly lines to make room for the vertical diffusion run which cannot be realized otherwise. We have not found an algorithm to find a sequence such that the number of such enlargements is minimized. Fig. 16 shows a layout with unrealizable diffusion runs. By permuting the rows, a realizable layout is obtained in Fig. 18. The layout of Fig. 19(a) is unrealizable regardless of the permutation of the rows. The spacing between gate C and D is enlarged to realize the vertical connection.

IX. IMPLEMENTATION

A procedure to obtain a layout $L(f, h)$ from a circuit description in terms of the net-gates sets $X(n_i)$ has been implemented. Though the procedure does not find the optimal solution in either stage one or two, the results are quite good and the time complexity seems acceptable. Figs. 9 and 10 show the layout of two circuits with 118 and 306 transistors, obtained in 9 and 28 s on a mainframe computer, respectively. Note that in Fig. 9 the gate spacing is enlarged at two places to realize two vertical diffusion runs indicated by “|” lines. The realizable diffusion runs are shown in “*” lines.

X. SUMMARY

We have presented an algorithm to obtain a layout of static CMOS circuits in the style of gate matrix. The algorithm is based on a formulation of the problem as a problem of finding an interval graph from a connection graph such that the largest dominant clique is minimized. Fast heuristics are used to generate many interval graphs and the best is selected as the basis for layout. Vertical diffusion runs which connect transistors on different rows are seen to pose realizability problems and we have devised a method to permute the rows such that the diffusion runs are all realizable.

It should be noted that in this paper we have not considered all of the practical constraints in the layout of a circuit. For example, it is frequently required that certain gate lines be designated as input and output terminals and their positions are fixed. In a situation where all the input lines are placed outside the left edge of the matrix and all the output line outside the right edge, the approach taken in [3] may be applied to the gate matrix case. Another practical consideration is that if the aspect ratio of the gate matrix is unacceptable, the matrix must be partitioned into several submatrices stacked in a serpentine fashion one on top of another. There arises the question of an optimum partition to minimize the total area, including the area taken up by the interconnections between submatrices. Thirdly, in our model, all the transistors are assumed to

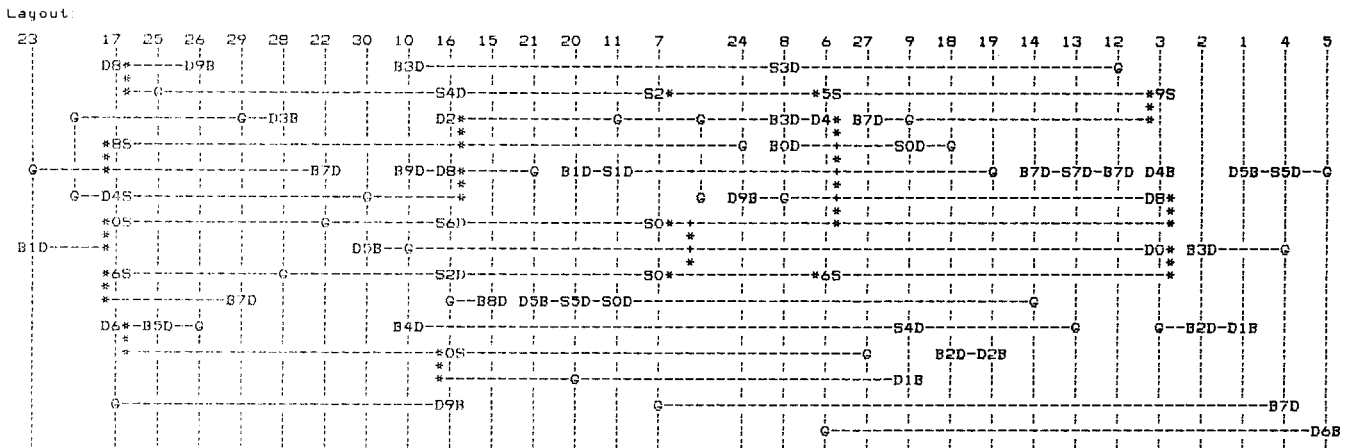


Fig. 9. The layout of n-part of Example 1.

be of the same size. If this is not the case, “big” transistors will have to be modeled as two or more parallel transistors connected by vertical diffusion runs. These and possibly other practical considerations will be the subject of a future paper.

APPENDIX A

CONSTRUCTION OF AN INTERVAL GRAPH FROM A CONNECTION GRAPH

It is best to explain the algorithm by means of an example. Consider the circuit of Figure 11. Its abstract representation is shown in Fig. 12. The gate-nets sets are tabulated below:

Gate g	$Y(g)$
A	{1, 2}
B	{2, 4}
C	{2, 3, 6}
D	{3, 4, 8}
E	{4, 7, 8}
F	{5, 6, 7}
G	{7}
H	{1, 3, 5}.

From this table, it is easy to construct the connection graph H shown in Fig. 12.

Let $A = [a_{ij}]$ be a matrix whose rows are the vertices (nets) of H and whose columns are the set of all dominant cliques of H . The element $a_{ij} = 1$ if and only if vertex i is contained in dominant clique j ; otherwise $a_{ij} = 0$. For the circuit at hand, we have

	1	2	3	4	5	6	7	8	
1	1	0	1	0	0	0	0	0	
2	1	1	0	1	0	0	0	0	
3	1	1	1	1	1	0	0	1	
4	0	1	0	0	1	0	1	0	= A
5	0	0	1	0	0	1	0	1	
6	0	0	0	1	0	1	0	1	
7	0	0	0	0	0	1	1	0	
8	0	0	0	0	1	0	1	0	

In practice the degree of the vertices of H is small relative to the number of vertices and we have found that the enumeration is not a major consumer of computer time even for a circuit with 232 transistors and 144 nets.

Matrix A is known as the vertex versus dominant clique matrix [7] and it is also shown in this reference that a given graph is an interval graph if and only if its vertex versus dominant clique matrix has the property that the columns can be so arranged that in each row, the ones appear consecutively.

In our case, the connection graph may or may not be an interval graph and the matrix A may or may not have the consecutive ones property. For our purpose, we note that matrix A can always be made to have the consecutive ones property by replacing all the zeros by ones (fill-ins) between the leftmost and rightmost ones in each row. The resultant matrix corresponds to an interval graph whose largest dominant clique has a size equal to the largest number of ones in any column. Problem 1 of Section III can now be stated as:

Given the vertex versus dominant clique matrix of a connection graph, find a permutation of the columns such that when zeros are replaced by ones in each row to produce the consecutive ones property, the largest number of ones in any column is minimized.

The computation complexity of this problem appears to be $N!$, where N is the number of dominant cliques. However, instead of attempting to find the optimum solution, we shall use a fast heuristic algorithm to generate many solutions and pick the best among them. Our strategy is to reconstruct matrix A column by column in such a way that as each column is added the increase in size of the largest dominant clique generated thus far is minimized.

Let the reconstructed matrix be A' . Let the cliques of H (columns of A') be denoted as C_i . Let $\text{Adj}(C_k)$ be the set of cliques which have one or more vertices in common with C_k . Let CL and CR be the leftmost and rightmost columns of the partially constructed A' , respectively.

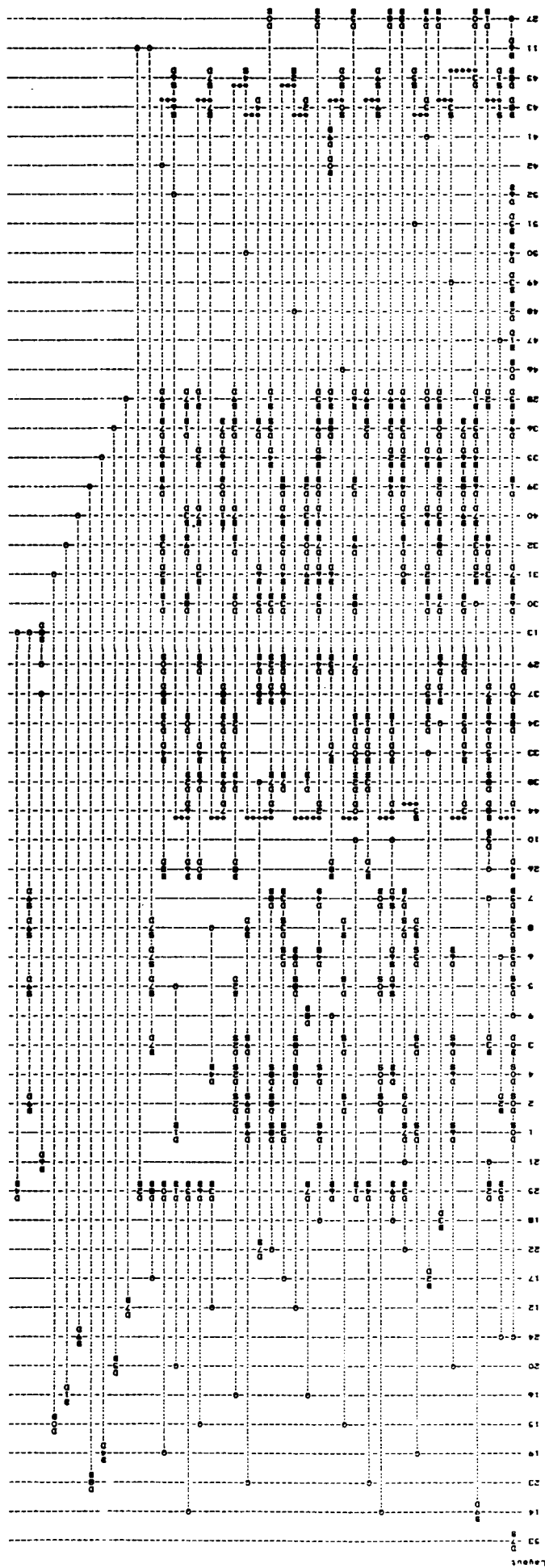


Fig. 10. The layout of n-part of Example 3.

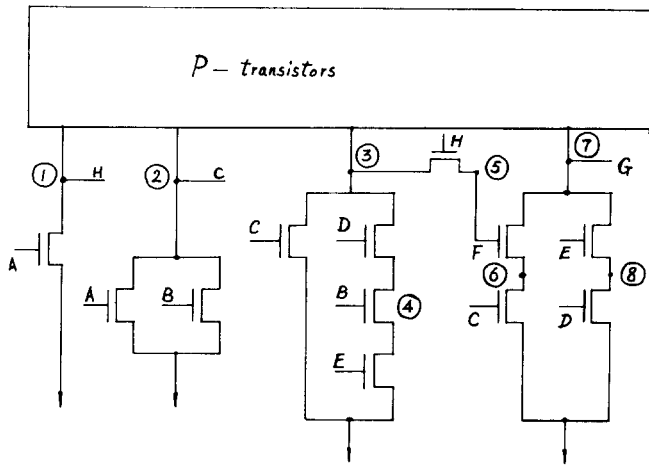


Fig. 11. n-part of a CMOS circuit.

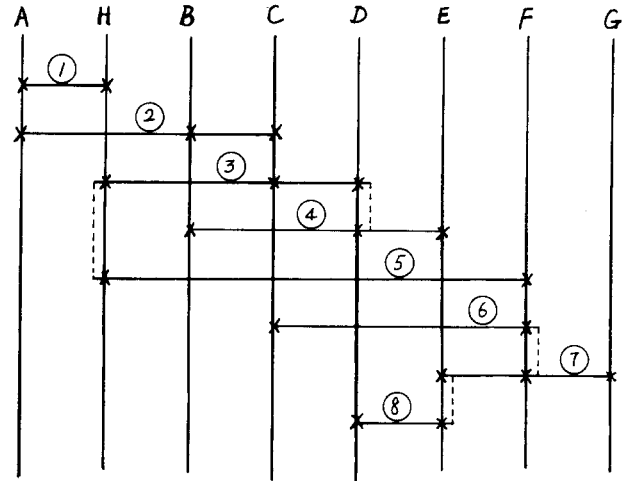


Fig. 12. Abstract representation and connection graph of the circuit in Fig. 11.

Let fill-in (i) be the number of fill-ins required to produce the consecutive ones property of the partial A' , if clique i is placed to the right of CR. Let $\text{com}(i)$ be the number of vertices common to clique i and CC the clique under consideration.

Further, let $\text{enlarge } R(i)$ ($\text{enlarge } L(i)$) be the increase in the size of the largest dominant clique of A' if clique i is placed to the right of CR (CL). Let M be the number of dominant cliques and N be the total number of trials desired.

Lastly, let SIZE be the largest number of ones in any

column of A' . It is also by definition the size of the largest dominant clique of A' . Let addone be the total number of fill-ins in A' .

Algorithm Interval Graph

Input: Matrix A .

Output: Matrix A' with the consecutive ones property.

keepsizes = a very large integer;

keepfill = keepsizes;

trials = 1;

repeat

if ($M < 50$) $k = \text{trials}$;

else $k = \text{a random number ranging from 1 to } M$;

$CC = CR = C_k = \text{starting column}$;

size = the size of C_k ;

addone = 0;

repeat

find Adj (CC);

if (Adj (CC) is not empty) {

choose $C_j \in \text{Adj} (CR)$ such that $\text{enlarge } R(j)$ is minimum;

if there are ties {

choose C_j among the ties such that fill-in (j) is minimum;

if there are ties {

choose C_j among the ties such that $\text{com}(j)$ is maximum;

if there are ties {

choose any among the ties; } } }

if ($\text{enlarge } R(j) < \text{enlarge } L(j)$) {

```

        place  $C_j$  to the right of CR;
        CR =  $C_j$ ; CC =  $C_j$ ; }
    else {
        place  $C_j$  to the left of CL;
        CL =  $C_j$ ; } }
    else {
        if (CC < > CL) CC = left neighbor of CC in  $A'$ ;
        else {
            choose  $C_j$  among the unplaced columns such that the
            size of  $C_j$  is maximum;
            place  $C_j$  to the right of CR;
            CR =  $C_j$ ; CC =  $C_j$ ; } }
        update size and addone;
    until all cliques are placed;
    if (size < keeptime) {
        keeptime = size;
        keep  $A'$ ; }
    else if (size = keeptime and addone < keepfill) {
        keepfill = addone;
        keep  $A'$ ; }
    trials = trials + 1;
until trials = min ( $M$ ,  $N$ );

```

Following the algorithm, we find A' to be:

	3	1	2	4	8	6	7	5	(Old column number)
	1	2	3	4	5	6	7	8	(New column number)
1	1	1	0	0	0	0	0	0	
2	0	1	1	1	0	0	0	0	
3	1	1	1	1	1	x	x	1	
4	0	0	1	x	x	x	1	1	
5	1	x	x	x	1	1	0	0	= A'
6	0	0	0	1	1	1	0	0	
7	0	0	0	0	0	1	1	0	
8	0	0	0	0	0	0	1	1	

in which fill-ins are denoted by x 's.

Fig. 13 shows the results of 90 trials of the reconstruction of A' of a larger example. Matrix A has 100 columns and the starting column is randomly chosen. The results suggest that it matters which column is the starting column, for the maximum clique sizes range from 34 to 55 for a circuit with 232 n-transistors, 126 gates, and 144 nets. The time taken to generate all 90 solutions was 12.5 s.

APPENDIX B

DETERMINATION OF GATE ASSIGNMENT FUNCTION f

The reconstructed matrix A' has the consecutive ones property and therefore corresponds to an interval graph $I(L)$, where L is some layout to be determined. In A' each row (net) of ones is an interval and the overlapping characteristics of the intervals of $I(L)$ are explicitly shown in A' .

Following [3], we let l_i (r_i) be the column number of the leftmost (rightmost) one of row i . We define the "interval" of row i as $[l_i, r_i]$. Then gate g_k in the layout L must lie within the interval $[L_k, R_k]$ determined from

$$[L_k, R_k] = \bigcap_{n_i \in Y(g_k)} [l_i, r_i].$$

For example, consider gate C of the example of Appendix A. With reference to the gate-nets sets, we find $Y(C) = \{2, 3, 6\}$, and from the reconstructed matrix A' , we have $[l_2, r_2] \cap [l_3, r_3] \cap [l_6, r_6] = [2, 4] \cap [1, 8] \cap [4, 6] = [4, 4]$. The complete list is

Gate	$[L_k, R_k]$	f
A	[2, 2]	2
B	[3, 4]	3
C	[4, 4]	4
D	[7, 8]	8
E	[7, 7]	7
F	[6, 6]	5
G	[6, 7]	6
H	[1, 2]	1.

The intervals $[L_k, R_k]$ specify the relative positions of the gates and the assignment function f is derived from the set of intervals as indicated above.

Once the gate assignment is determined, we apply the left-edge first algorithm of Hashimoto and Stevens [8] to the intervals in A' to find the net assignment function h , and the net-to-gate connections are made according to the net-gates sets $X(n_i)$. For the example of Appendix A, the function h is given below, listed in the order in which the nets are picked according to the left-edge algorithm.

Net n_i	$h(n_i) =$ row
3	1
5	2
8	2
1	3
4	3
2	4
7	4
6	5

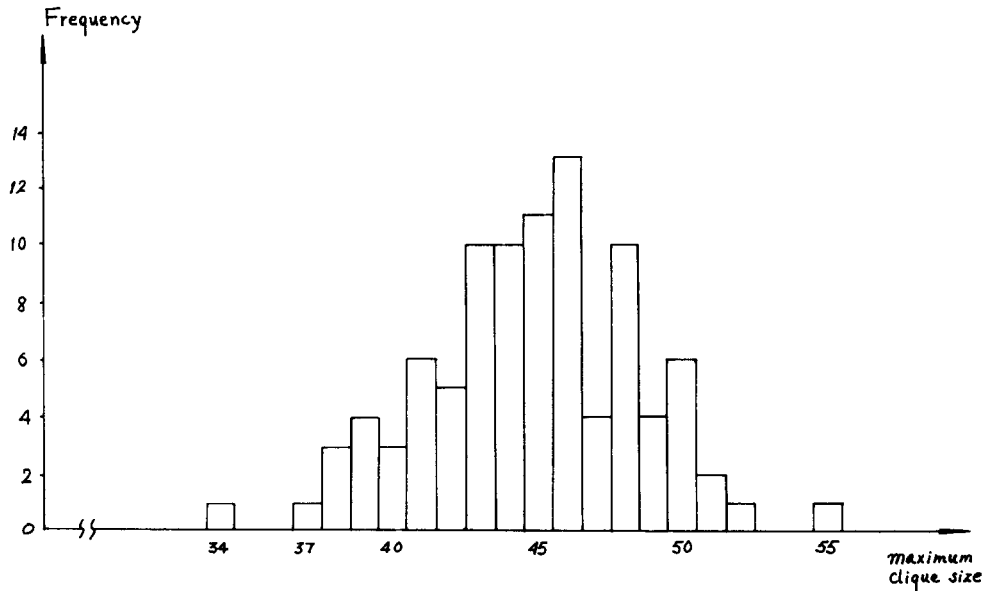


Fig. 13. Histogram of the result of algorithm INTERVAL GRAPH. Number of trials: 90; number of cliques: 100; number of nets: 144.

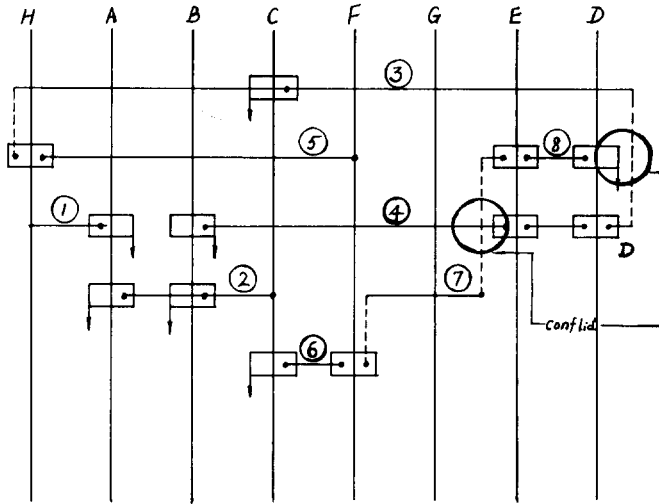


Fig. 14. Layout of the circuit in Fig. 11.

The layout $L(f, h)$ is shown in Fig. 14 after we have identified transistor D as the end transistor in the series-connected net 4.

If we place the vertical diffusion runs onto the layout, we find that $D(3, 4)$ and $D(7, 8)$ are not realizable. However, after merging nets 6, 7, and 8 and placing them in the row of net 6, the layout becomes realizable.

APPENDIX C REALIZABLE NET ASSIGNMENT

Suppose a gate assignment function f is found and the layout $L(f, h^*)$ is obtained with the net assignment function h^* determined without regard to the vertical diffusion runs. If h^* is realizable, then $L(f, h^*)$ is the layout; otherwise, we proceed to find a permutation of the rows such that all the diffusion runs are realizable.

The vertical runs between rows are represented by the adjacency graph $J(V, E)$ as defined in the text. Our goal

is to find an acceptable sequence of the rows from the sequence specified by h such that the layout is realizable.

We define a violation function $v(i, k, j)$ with respect to three rows i, j , and k as follows:

$$v(i, k, j) = n$$

if for the sequence $\langle i, k, j \rangle$ there are n -transistors or gate connections in row k that conflict with the diffusion runs between row i and row j .

We say that a sequence of rows $\langle r_1, r_2, \dots, r_M \rangle$ is *realizable* if for all $i < k < j$, $v(r_i, r_k, r_j) = 0$. Our plan is to construct a realizable sequence vertex by vertex by adding a vertex to a partial sequence if it causes no violation. In general, the adjacency graph will have cycles. We first find a partial sequence for each "cycle group" and then insert the remaining vertices into the partial sequences to obtain the complete sequence.

We define a *cycle group* as a set of vertices in which every vertex belongs to a cycle or to a set of cycles in which every pair of cycles have at least two vertices in common. We define an *extension* as a path which begins at a vertex of a cycle group and terminates on a vertex with degree one or on a vertex of another cycle group. Figure 15 shows an adjacency graph in which $\{2, 3, 4, 5\}$ and $\{8, 9, 10\}$ are cycle groups and $\{1, 2\}$, $\{5, 8\}$ and $\{5, 6, 7\}$ are extensions.

We start a partial sequence of a cycle group with the vertex having the longest extension (source) and end with a vertex with the second longest extension (sink). After a vertex is added to a partial sequence S_q , the next vertex is chosen from a set of "acceptable candidates" defined as follows:

First we define the set of candidates $K = \{x_i \mid x_i \in V - S_q \text{ and } x_i \text{ is adjacent to some } v_j \in S_q\}$. A candidate x is *acceptable* if $v(v_i, x, y_k) = 0$ for all i, k , where $v_i \in S_q$ and $y_k \in V - S_q - x$. The *acceptable candidate set* $K_1 =$

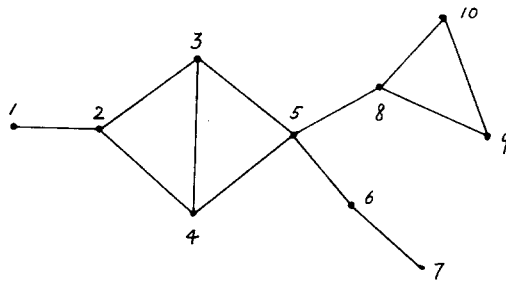


Fig. 15. Example of an adjacency graph.

$\{x_i \mid x_i \text{ is acceptable}\}$ and the *unacceptable candidate set* $K_2 = K - K_1$.

Associated with each acceptable candidate is a *risk function* which signifies that if this candidate is not selected as the next vertex in the partial sequence, then any other candidate which comes before it will introduce one or more violations. More specifically, the risk function of a candidate x chosen from a candidate set K' is defined as

$$R(x, K') = \sum [v(v_i, w_j, x) + v(w_j, x, y_k)]$$

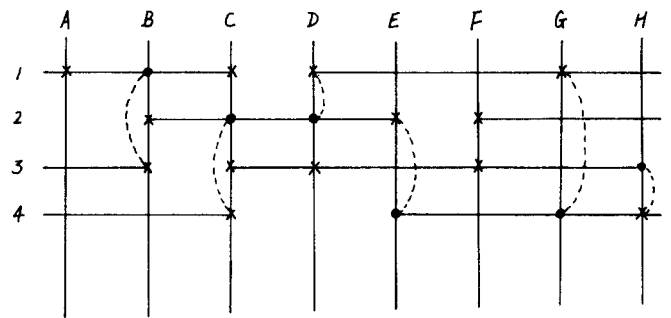
the summation being taken over all i, j, k , and where $v_i \in S_q$, $w_j \in K' - x$, and $y_k \in V - S_q - K'$. In this formula, x is the vertex under consideration, v_i is a vertex of the partial sequence, w_j is a candidate other than x , and y_k is one of the remaining vertices. The term $v(v_i, w_j, x)$ gives the total violations introduced if we select one of the other candidates w_j , to be placed before x . The second term $v(w_j, x, y_k)$ gives the violations on row x introduced by the diffusion runs between w_j and some remaining vertices. Note that if K' is an acceptable set, the first term is zero.

We select an acceptable candidate x from K_1 , to be added to the partial sequences S_q according to the following order of priority:

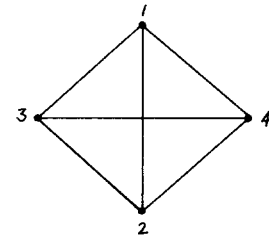
- 1) $\sum_i d(x, v_i)$ is maximum, where $v_i \in S_q$ and $d(k, y)$ is the total number of diffusion runs between rows x and y according to the net assignment function h ;
- 2) $R(x, K_1)$ is maximum;
- 3) the shortest distance between x and the sink is minimum; and
- 4) $|K(S_q + x)|$ is maximum, where $K(S_q + x)$ is the set of candidates after we place x into S_q .

As an example, consider the layout depicted in Fig. 16. It is not realizable because of the vertical diffusion runs along columns B and C . The adjacency graph is shown in the same figure and it does not have any extension. In Fig. 17, we give the steps leading to an acceptable sequence $\langle 1, 3, 2, 4 \rangle$ and Fig. 18 shows the layout, which is realizable.

It is possible that at some point in the construction of the partial sequence, all candidates are unacceptable, i.e., K_1 is empty. We then select a candidate y and for each violation induced by y we enlarge the spacing between gate lines to place the diffusion run so as to avoid the conflict with a transistor or a gate connection. The selection is made in accordance with the following priority condi-



(a)



(b)

Fig. 16. (a) An example of layout. (b) Its adjacency graph.

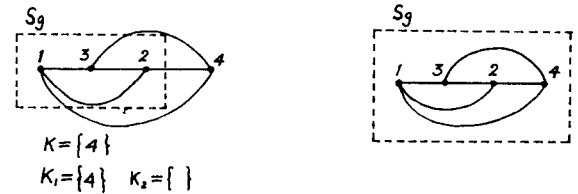
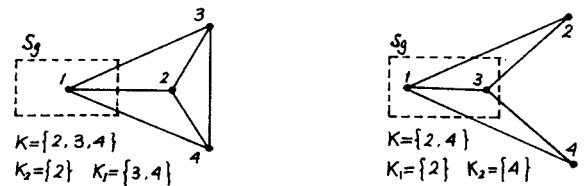


Fig. 17. Procedure to find an acceptable sequence.

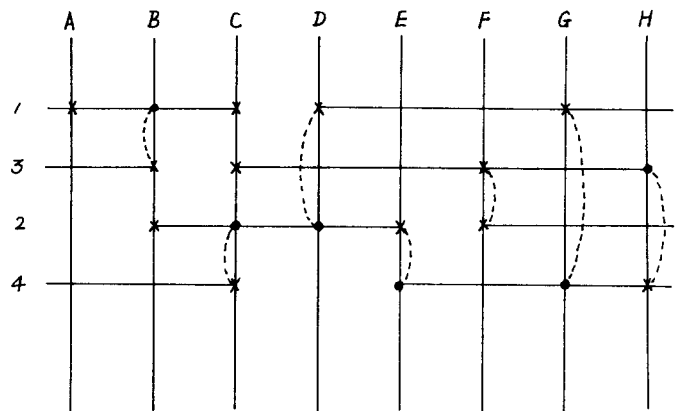


Fig. 18. Resultant realizable layout.

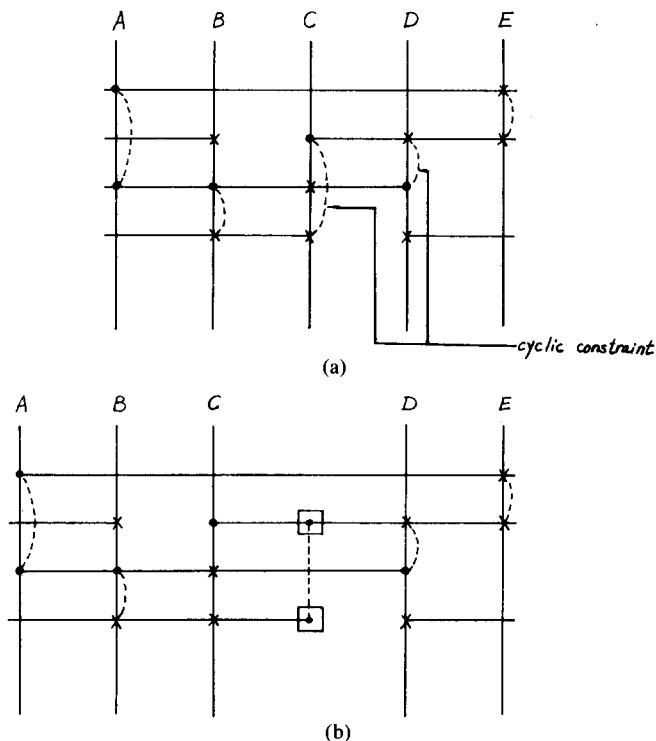


Fig. 19. (a) Unrealizable layout. (b) Layout by enlarging the gate spacing.

tions:

- 1) $\sum_i d(y, v_i)$ is minimum, where $v_i \in S_q$;
- 2) $R(y, K_2)$ is maximum;
- 3) the shortest distance between y and the sink is minimum; and
- 4) $||K(S_q + y)||$ is maximum.

In the risk function of 2), y is an unacceptable candidate and the first term in the risk function is not zero.

Fig. 19 shows a situation in which after rows 1 and 2 are added to the partial sequence, all the candidates are unacceptable. In this case, we enlarge the spacing between gates C and D to place the diffusion run between rows 2 and 4. The enlargement will increase the total area of the layout. However, since in practice, the number of gates is usually much greater than the number of rows, this increase is tolerable.

ACKNOWLEDGMENT

The test examples are supplied by Dr. Joel Schoen of IT&T whose encouragement and interest in our work are deeply appreciated.

REFERENCES

- [1] A. D. Lopez and H-F S. Law, "A dense gate matrix layout method for MOS VLSI," *IEEE Trans. Electron Devices*, pp. 1671-1675, Aug. 1980.
- [2] O. Wing, "Automated gate matrix layout," in *Proc. 1982 IEEE Int. Symp. Circuits and Systems*, Rome, Italy, pp. 681-685, 1982.
- [3] T. Ohtsuki, H. Mori, E. S. Kuh, T. Kashiwabara, and T. Fujisawa, "One-dimensional logic gate assignment and interval graphs," *IEEE Trans. Circuits Syst.*, pp. 675-684, Sept. 1979.
- [4] T. Ohtsuki, H. Mori, T. Kashiwabara, and T. Fujisawa, "On minimal augmentation to a graph to obtain an interval graph," *J. Computer Syst. Sci.*, pp. 60-97, Feb. 1981.
- [5] T. Kashiwabara and T. Fujisawa, "An NP-complete problem on interval graphs," in *Proc. 1979 IEEE Int. Symp. Circuits and Systems*, Tokyo, Japan, pp. 82-83, July 1979.
- [6] O. Wing, "Interval graph based gate matrix layout," in *Proc. 1983 IEEE Int. Conf. CAD*, Santa Clara, CA, Sept. 1983.
- [7] D. R. Fulkerson and O. A. Gross, "Incidence matrices and internal graphs," *Pacific J. Math.*, no. 3, pp. 835-5, 1965.
- [8] A. Hashimoto and J. Stevens, "Wire routing by optimizing channel assignment within large apertures," in *Proc. 18th Design Automation Workshop*, pp. 155-168, 1971.
- [9] J. T. Li, "Algorithms for gate matrix layout," in *Proc. 1983 IEEE Int. Symp. Circuits Syst.*, Newport Beach, CA, pp. 1013-1016, 1983.
- [10] T. Asano, "An optimum gate placement algorithm for MOS one-dimensional arrays," *J. Digital Syst.*, vol. VI, pp. 1-27, 1981.
- [11] C. Berge, *The Theory of Graphs*. London, England: Methuen, 1962, p. 36.

*

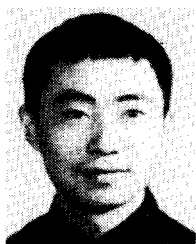


Omar Wing (S'50-A'53-M'58-SM'68-F'73) received the B.S. degree from the University of Tennessee, Knoxville, in 1950, the M.S. degree from the Massachusetts Institute of Technology, Cambridge, in 1952, and the Eng.Sc.D. degree from Columbia University, New York, in 1959.

From 1952 to 1956 he was Member of the Technical Staff at Bell Telephone Laboratories. Since 1959 he has been a member of the faculty of the Department of Electrical Engineering, Columbia University, where he is currently Professor and Chairman. He was a Fulbright Lecturer at the Institute of Electronics, Chiao-Tong University, Taiwan, in 1961, a Ford Foundation Engineering Resident at the Thomas Watson Research Center, IBM in 1966-1967, a Guest Professor at the Technical University of Denmark in 1973, and a Fulbright Senior Lecturer at the Technical University of Eindhoven, the Netherlands in 1979-1980. Other visiting appointments include one at the Indian Institute of Technology, Kanpur, India, in 1978; at the South China Institute of Technology, Guangzhou, China, in 1979; at the Technical University of Istanbul, Turkey, in 1979; at the Technical University of Lisbon, Portugal, in 1979; and at the Jiao Tong University at Shanghai, China, in 1982. His current research interests include VLSI CAD, circuit simulation, and parallel computation.

Dr. Wing served as Editor of the IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS from 1974 to 1978 and as President of the IEEE Circuits and Systems Society in 1978. He is a recipient of the IEEE Centennial Medal.

*



Shuo Huang was born in Beijing, China on February 28, 1956. He received the B.S. degree in electronics from Beijing Normal University, Beijing, China in 1982 and the M.S. degree in electrical engineering from Columbia University, New York.

Since 1982, his research interests have focused on computer-aided design of integrated circuits, with particular emphasis on layout.

*



Rui Wang received the B.S. degree in electrical engineering from South China Institute of Technology, Guangzhou, China, in 1982 and M.S. degree from the Department of Electrical Engineering, Columbia University, New York, NY, in 1984. She is currently working towards the Ph.D. degree in electrical engineering at Columbia University.

She is a Research Assistant in the Electrical Engineering Department of Columbia University where her current interests include VLSI layout automation and circuit simulation.